DB Monitoring: стратегия деплоя и отката

Домашнее задание B10 «Стратегии деплоя и релиз-процессы» (слайд 45; практика — слайды 33–38), Issue #14. Сверено с `master` `f891234` на 10.11.2026. Проектное решение: переключатель трафика и автоматическое сравнение версий не реализованы, откат не испытан; «100 % трафика» — целевое состояние схемы.

1. Что выпускается

Приложение — Flask со встроенным планировщиком: сбор метрик (micro-batch, по умолчанию раз в 15 мин), оценка аномалий, уведомления, ночное обучение. Compose поднимает PostgreSQL-источник, TimescaleDB-хранилище, Flask и по профилю `inference` FastAPI-инференс без состояния. Выпускаются два независимых артефакта: **образ приложения** (тег `X.Y.Z`, digest, `ENV APP_VERSION=vX.Y.Z`) и **версия модели** Isolation Forest в реестре (`if-<SHA-256>`, схема признаков `1.0.0`). Версия модели включается для каждого источника — привязкой (`bindings`) адаптера. Основной поток работы — тики сборщика, а не HTTP-запросы.

Какой образ запущен. `/healthz.version` берётся из окружения контейнера; `init_compose_demo.py` пишет в `.env` `APP_VERSION=<SHA>`, и через `env_file` он перекрывает `ENV` образа, поэтому при переходе на релизный образ строку удаляют из `.env` и при откате не возвращают. Образ подтверждает digest: `RepoDigests` тега содержит digest из записи релиза, `docker inspect -f '{{.Image}}' <контейнер>` = `Id` образа; версия — лишь дополнительный признак.

2. Выбор: blue-green для приложения, shadow → canary для модели

Приложение — blue-green как целевой подход; на стенде — проверка green и последовательная замена. Ошибка кода проявляется сразу (`/healthz`, 5xx, сбой сборщика), поэтому важнее быстро вернуть рабочую версию, чем постепенно набирать уверенность (вопрос слайда 38). Классический blue-green требует двух готовых сред и переключателя трафика. На стенде их нет, и вторую среду Flask нельзя держать запущенной: планировщик стартует в каждом процессе без отключающей настройки (двойной сбор и обучение), а миграции хранилища требуют остановить старые процессы (migrations.md). Доступный способ: green проверяется в изолированном Compose-проекте на копии данных, затем рабочий контейнер заменяется между тиками с простоем порядка минуты (оценка). Прежний образ — артефакт для отката, а не работающая среда: возврат — повторная замена, а не мгновенное переключение.

Модель — shadow, затем canary по источникам. Ошибка модели «тихая» (слайд 12): сервис работает, но хуже находит инциденты. Нужны проверка на реальных данных и сравнение с предыдущей версией. Основа есть: версия задаётся для каждого источника, оценки хранятся с ключом версии, откат — возврат привязки без переобучения. Единица «трафика» — источник (подключение + схема + таблица).

Отклонено. Canary приложения: нет маршрутизатора, у внутреннего инструмента мало пользователей, две версии делили бы хранилище и фоновые задания. Blue-green модели «сразу на все источники»: быстрый откат уже дают привязки, а одновременное переключение скрывает ухудшение до пропущенных инцидентов.

3. Схема процесса: от «код готов» до «100 % трафика»

[P] — реализовано; [P*] — реализовано, но здесь не запускалось; [П] — предлагается.

```
КОД ГОТОВ: PR в master
  ▼
1 CI [P]: Ruff; pytest 3.11–3.13; integration со сборкой образа --no-cache; Compose-сценарий
  (technical_pass + check-outcome); после слияния — E2E на master ──────── провал ──► СТОП, исправить
  ▼ успех
2 Тег vX.Y.Z → release.yml [P*]: образ :X.Y.Z (и :vX.Y.Z), latest, previous; APP_VERSION=vX.Y.Z.
  Записать digest нового и прежнего образов, прежний тег, снимки /metrics; разворачивать :X.Y.Z
  ▼
3 GREEN [П]: свежий бэкап → восстановление в отдельный Compose-проект → новый образ один раз
  (миграция) → инварианты БД, /healthz, digest, smoke, тик сборщика, /v1/model привязок
  ──────────────────────────────────────────────────────── провал ──► СТОП, рабочая версия не тронута
  ▼ успех
4 ЗАМЕНА между тиками [П; команды Compose есть]: stop app → бэкап и снимок схемы БД (столбцы, ключи, индексы) →
  без APP_VERSION в .env: APP_IMAGE=<образ>:X.Y.Z docker compose up -d --no-build app → digest (§1)
  ▼
5 КОНТРОЛЬ 30 мин, затем 24 ч [П]: строки 1–4 таблицы §4; порядок: откат → держать → продолжить
  ├─ условие отката (аварийное — сразу, при любом объёме данных) ──► ОТКАТ приложения (§5 А)
  ├─ условие «держать» или мало данных ──► ДЕРЖАТЬ, продлить окно
  ▼ норма
  ПРИЛОЖЕНИЕ ПРИНЯТО (100 % трафика с шага 4); привязки моделей не менялись
  ▼ отдельный релиз модели
6 Допуск реестра [P], независимый test: event_recall ≥ 0,8; FPR ≤ 0,1;
  медиана задержки резких событий ≤ 1 тик ──────────────────────── не допущена ──► СТОП
  ▼ допущена
7 SHADOW [П]: ≥ 96 общих наблюдений на источник, без уведомлений; строки 5–8 ──────── хуже ──► СТОП
  ▼ не хуже
8 CANARY по N привязкам [P: привязки; П: процедура]: 1 источник → ⌈N/2⌉ → N, шаг ≥ 96 наблюдений;
  совпадающие шаги пропускаются. По строкам 5–8: продолжить / ДЕРЖАТЬ / ОТКАТ привязок (§5 Б)
  ▼
100 % ТРАФИКА: образ X.Y.Z и новая модель у всех источников; прежние образ и версия модели
хранятся не менее 7 дней как точка отката
```

Реализовано и проверялось (CI, тесты, протоколы)	Есть в коде, но здесь не запускалось	Нет — предлагается этим документом
CI `tests.yml`/`lint.yml` и Compose-сценарий #13; реестр и допуск `admitted()` #5; привязки и оценки с ключом версии #12; `/healthz`, `/metrics`, журнал `collector_runs`, алерты `HttpErrorRatioHigh` #15 и `ModelAnomalyRateHigh` #16; backup/restore (тесты)	`release.yml`, `promote.yml` — тегов `v*.*.*` нет; runbook отката — dry-run не зафиксирован; процедура миграций на рабочих данных	Две обслуживающие среды и переключатель трафика; фоновая shadow-оценка (ручная оценка новейшей точки неактивной версией подавляет её уведомление) и сравнение версий; Prometheus-сервер; допущенная модель (её нет, §6); внешняя доставка алертов

4. Критерии успешности и пороги

Сравнение с предыдущей версией: приложение — с окном той же длины до релиза (разность двух снимков `/metrics` за 30 мин до замены: счётчики обнуляются при замене, Prometheus-сервера нет), модель — на тех же наблюдениях. Правила #15 (10 %, ≥ 10 запросов) и #16 (k из 20) есть в коде, остальные пороги **проектные**.

Порядок решения: откат → держать → продолжить. Условие «Откат» — откат; доли и p95 — при набранном минимуме, аварийные — сразу: 3 неуспешные пробы подряд (и при `down` компонента), 5xx на smoke, `model_unavailable` новой версии, чужой digest, пропущенный подтверждённый инцидент. Иначе «Держать» или мало данных — держать, не продвигать; «Продолжить» — только по всем строкам этапа.

№	Показатель и источник	Единица, окно	Продолжить	Держать	Откат
1	Доступность: `/healthz`, HEALTHCHECK раз в 30 с	доля успешных проб; 30 мин	100 %; digest и `version` релиза	≥ 1 неуспешная, не 3 подряд	3 подряд (`unhealthy`) или чужой digest
2	HTTP 5xx: `http_requests_total` без `/metrics`	%; 5 мин, ≥ 10 запросов	≤ 1 % и ≤ доли до релиза + 1 п. п.	> 1 % и ≤ 10 %	> 10 % (`HttpErrorRatioHigh`) или 5xx на smoke
3	Время ответа p95: `http_request_duration_seconds`	с; 15 мин, ≥ 30 запросов	≤ 1,2 × p95 до релиза и ≤ 2 с	прочее	два окна подряд > 2 × до релиза или > 2 с
4	Сбор: журнал `collector_runs` по подключениям	плановые запуски; 2 тика	`success`/`warning` у подключений, успешных до релиза	один `failed` или пропуск тика	2 `failed` подряд у одного из них или 2 пропуска тика
5	Ошибки инференса: `inference_predictions_total`, `outcome ≠ ok`	% вызовов; до ≥ 20 вызовов (1 источник ≈ 5 ч)	≤ 1 %	> 1 % и ≤ 5 %	> 5 % или `model_unavailable` новой версии
6	Качество и свежесть входов: `inputs.conclusive`	да/нет; ≥ 20 векторов	`true`	`false`	сбой сбора после релиза — строка 4
7	Доля аномалий против baseline той же версии: `ModelAnomalyRateHigh`	аномалий среди 20 последних	< k по baseline версии (≥ 500 норм. наблюдений); 3 из 20 — только demo #13	≥ k или нет baseline	только вместе со строкой 8
8	Расхождение с предыдущей версией на тех же наблюдениях	наблюдения; ≥ 96 на источник (≈ 24 ч)	≤ 2 новых флагов; флаги на подтверждённых инцидентах сохранены	> 2 или не разобраны	пропущен подтверждённый инцидент; firing строки 7 только у новой версии

Обоснование: 1 % — в норме 5xx около 0 (#15); 1,2 × — запас на шум и корзины гистограммы; 96 наблюдений — сутки сбора; «новый флаг» — аномалия у новой версии при норме у старой, 2 из 96 — порядок FPR demo-модели на test (1,66 %). Precision/recall/FPR — только на размеченном test (этап 6), а не по потоку.

5. План отката

Запуск и решение. Условие «Откат» или порча данных. Приложение откатывает ответственный за релиз (DevOps, M4 MLOps-roadmap), модель — ML-инженер; инциденты строк 7–8 подтверждает инженер данных.

А. Приложение. Цели (плановые, не измерены): обслуживание ≤ 10 мин, с БД ≤ 2 ч; порядок не испытан. (1) Зафиксировать время, digest, `/healthz.version`, снимок `/metrics`, журналы. (2) Функция под флагом (`FF_FORECAST`, `FF_LLM_NOTIFICATIONS`): правка `.env`, затем `APP_IMAGE=<образ>:X.Y.Z docker compose up -d --no-build --force-recreate app` (`restart` не перечитывает `env_file`, без `APP_IMAGE` образ берётся из `.env`); digest, `/healthz` 200, `/api/forecast/…` → 404; иначе — шаг 3. (3) `docker compose stop app` (веб, планировщик, сборщики); CLI с доступом к хранилищу не запускать. (4) Схема БД совпадает со снимком этапа 4 — совместима с прежним образом, шаг 6; иначе или без снимка — прежний образ на эту БД не запускать (migrations.md), шаг 5. (5) Дамп текущей БД (в ней и данные после бэкапа); бэкап этапа 4 — в новую БД того же TimescaleDB, сверка строк и ключей, проверка прежним образом в изолированном проекте; затем без подключений `metrics` → `metrics_failed`, копия → `metrics` (DSN в `environment` Compose не меняется). Данные после бэкапа — только в `metrics_failed`: их отсутствие согласуют до переключения, перенос — отдельной проверенной процедурой. (6) `APP_IMAGE=<образ>:<прежний X.Y.Z> docker compose up -d --no-build app` (тег с этапа 2, не `previous`) — запись возобновляется.

Б. Модель (цель ≤ 1 тик, 15 мин). Вернуть в `bindings` прежние `model_version` и `feature_schema_version` — без перезапуска: файл читается при каждом обращении (по коду, не испытано); версии реестра не удаляются. Несовпадение схемы признаков или scikit-learn — `model_unavailable`: при откате образа сначала `/v1/model`.

В. Обработанные запросы и результаты. Запросы при замене получают ошибку и повторяются; пропущенный тик не заполняется. Ключ оценок — «источник + время + версия + схема» (`ON CONFLICT DO NOTHING`): результаты новой версии остаются для разбора, прежняя досчитывает недостающие точки без повторных уведомлений. Уведомления не отзываются, их список — в запись инцидента; метрики не удаляются.

Г. Подтверждение. Обслуживание (≤ 10 мин): `healthy`, digest прежнего образа (§1), `/healthz` 200 с прежней версией 5 проб подряд, smoke без 5xx, условие `HttpErrorRatioHigh` ложно в первом окне 5 мин с ≥ 10 запросами (иначе — по smoke). Затем: 2 тика без `failed`; модель — по активной прежней версии (`/v1/model`, `outcome=ok` привязок, условие `ModelAnomalyRateHigh` её ряда ложно на полном окне). Отключённая версия наблюдений не получает, её `firing` в `resolved` не перейдёт (`app/model_monitoring.py`): в запись инцидента — вывод из эксплуатации, время, причина отката, последнее состояние ряда. `resolved` обязателен только для продолжающегося ряда (`HttpErrorRatioHigh`, прежняя модель) при условии восстановления.

6. Ограничения

- `release.yml` не запускался. В runbook и README: образ `…/db-monitoring` вместо `ghcr.io/<владелец>/<репозиторий>`; `docker pull …:previous && docker compose up -d app` без `APP_IMAGE` образ не сменит; `restart` не применяет `.env`; `/healthz?strict=true` при rate-limit `memory://` по коду всегда 503. Правка — отдельная задача.
- Допущенной модели нет (#9: FPR 13,27 %; demo #13 не допущена): этап 6 сейчас — «СТОП».
- Откат не испытывался; legacy IF без привязки перезаписывается ночью без версий — откатом не вернуть.